Global Currency Converter

A Java OOP Project Using Static Members

Java | Object-Oriented Programming

Submitted by

Name: Saurav Suman

ERP: RU-25-11317

Btech CSE (AI & ML)

Under the guidance of

Mr. Santanu Sasmal

IBM SME & Expert

Object Orientation Programming with Java

Project Overview

- Converts foreign currencies (USD, EUR, GBP) to Indian Rupees (INR)
- Built using Object-Oriented Programming concepts in Java
- Demonstrates practical use of static variables and static methods
- Eliminates need for object creation using utility class design

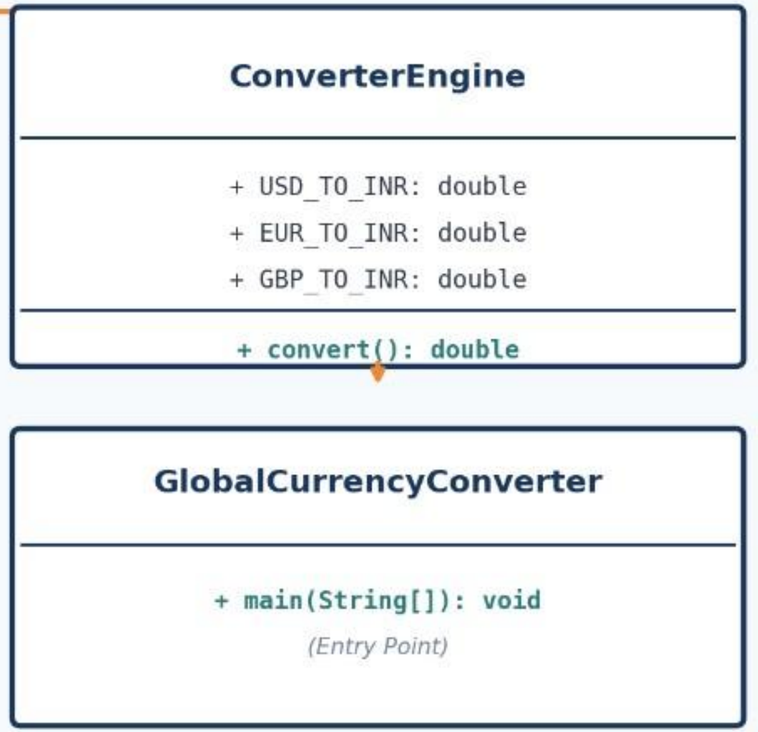
→ Key Objectives:

- Centralized management of exchange rates via static constants
- Memory-efficient conversion without object instantiation
- Reusable code structure for currency operations
- Simple command-line interface for user interaction

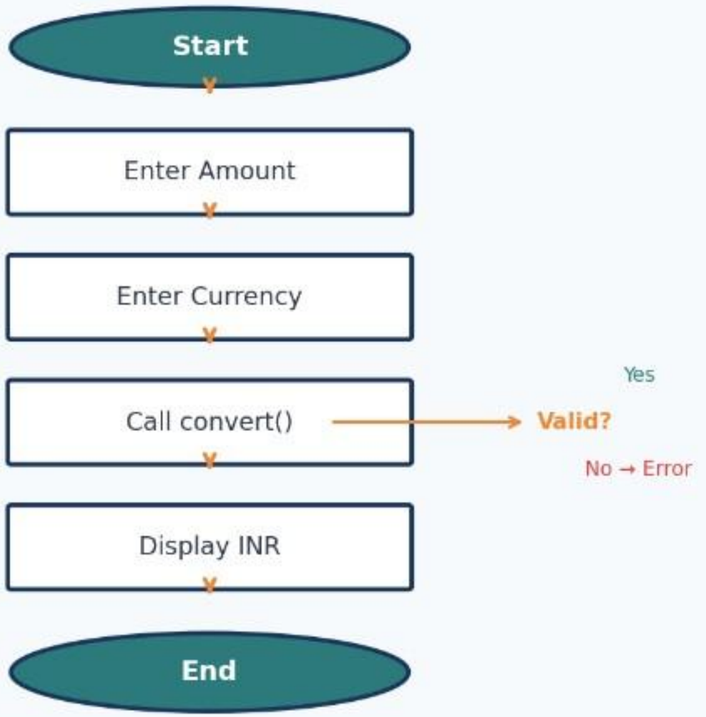
Code Implementation

```
public class ConverterEngine {
    // Static constants for exchange rates
    static final double USD_TO_INR = 83.12;
    static final double EUR_TO_INR = 90.45;
    static final double GBP_TO_INR = 105.30;

    // Static method for conversion (no object needed)
    public static double convert(double amount, String currency) {
        switch(currency.toUpperCase()) {
            case "USD": return amount * USD_TO_INR;
            case "EUR": return amount * EUR_TO_INR;
            case "GBP": return amount * GBP_TO_INR;
            default: return -1; // Invalid currency
        }
    }
}

public class GlobalCurrencyConverter {
    public static void main(String[] args) {
        double result = ConverterEngine.convert(100, "USD");
        System.out.println("100 USD = " + result + " INR");
    }
}
```

System Architecture



Program Flow



Conclusion

- Successfully demonstrates static members in Java OOP
- Static constants ensure centralized, consistent exchange rate management
- Static methods enable conversion without object instantiation
- Utility class design improves memory efficiency and code reusability

→ Why Static Members?

- Memory Efficiency: Single copy shared across all calls
- Code Reusability: Direct class-level access without objects
- Centralized Control: Rates managed in one location
- Better Performance: No overhead of object creation/destruction

→ Future Enhancements:

- Add more currencies (JPY, AUD, CAD)
- Real-time API integration for live rates
- GUI implementation using Java Swing/JavaFX